

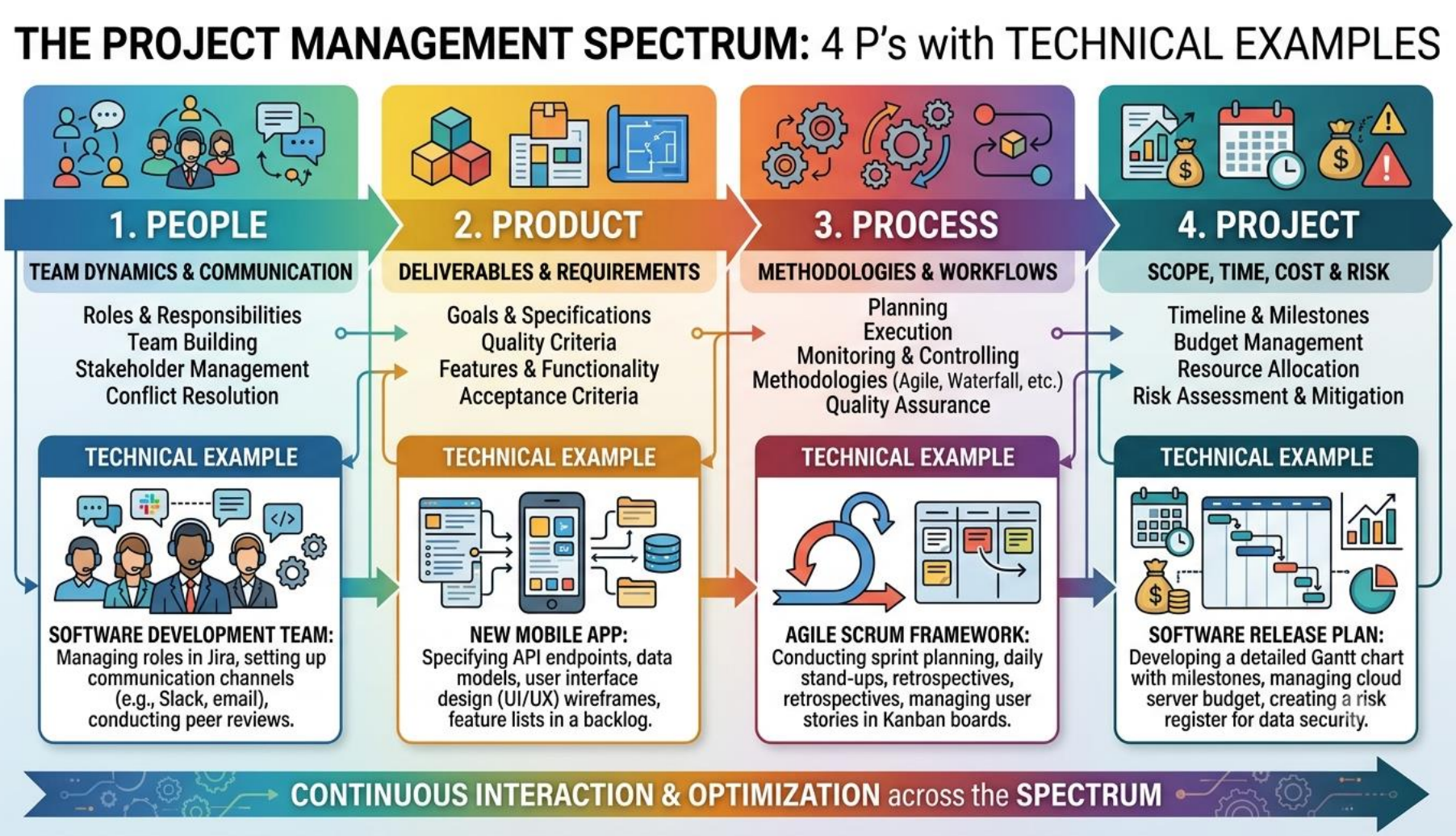
G.H. Raisoni College of Engineering and Management

Department of Information Technology

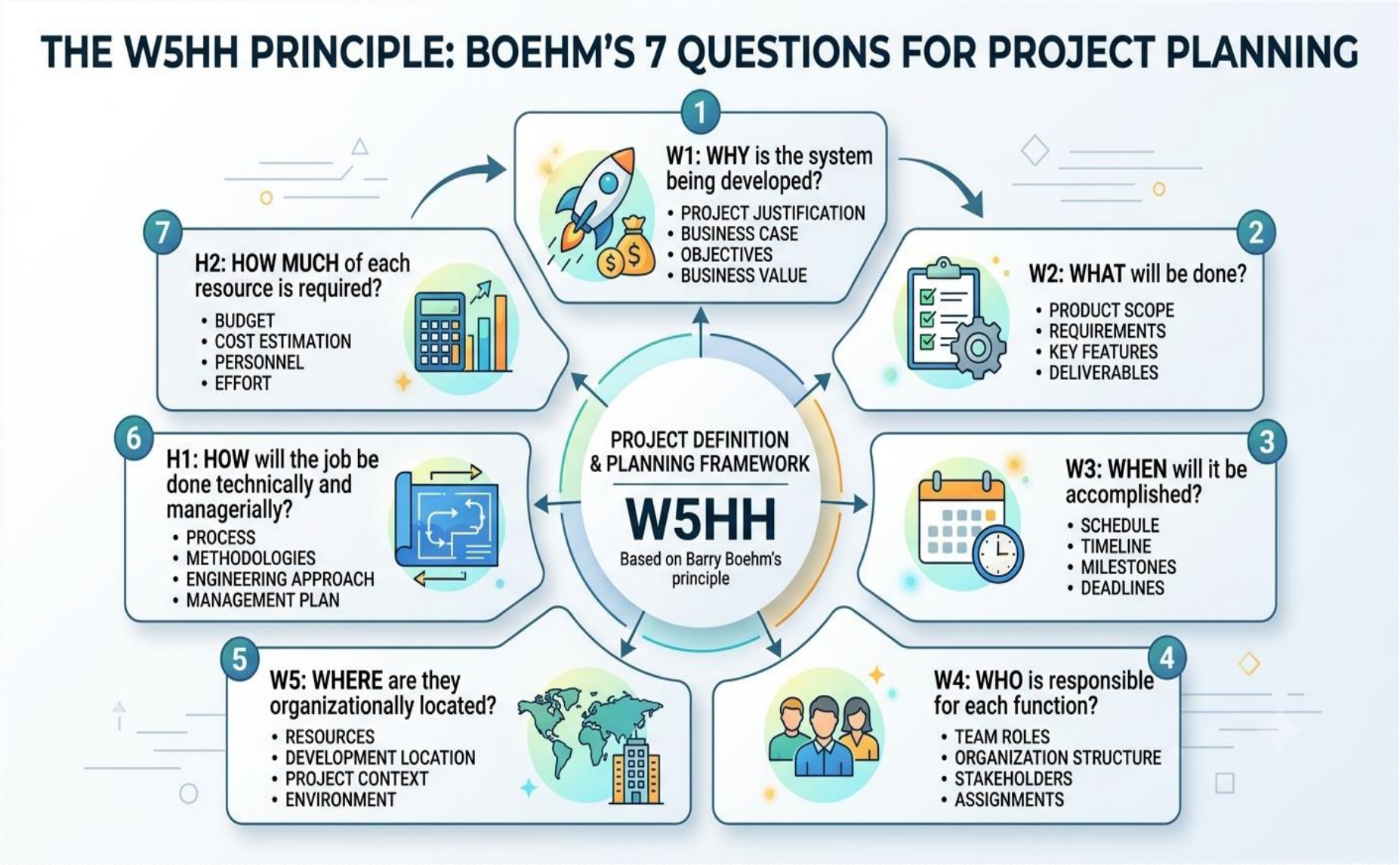
Unit 5 :PROJECT MANAGEMENT & ESTIMATION

Prof. Sagar Mahajan

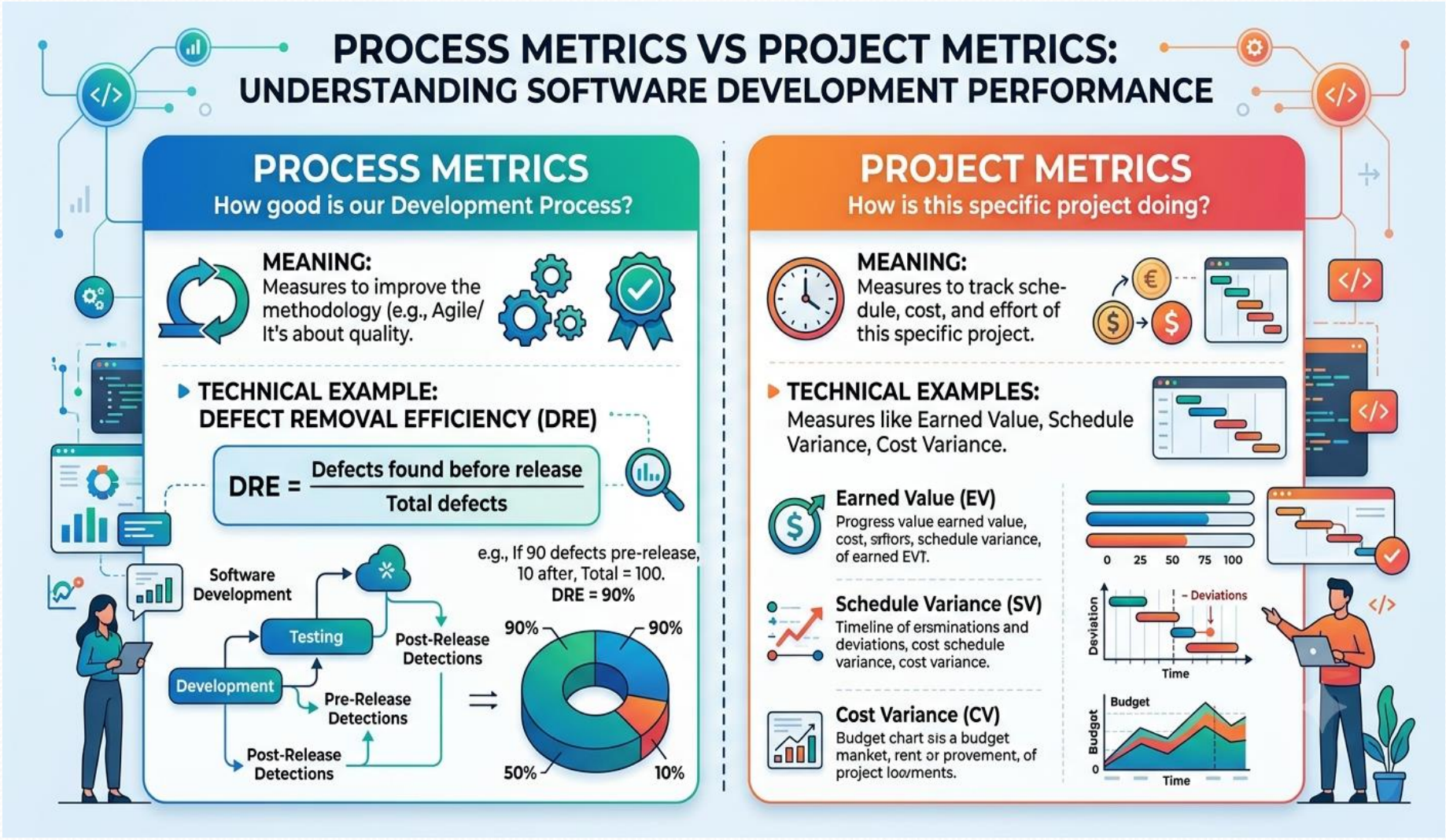
The Management Spectrum



The W5HH PRINCIPLES (Boehm's 7 Questions)



Metrics in Process & Project Domains



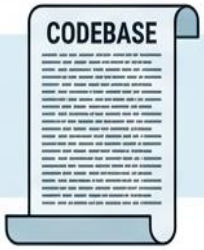
Software Measurement (Size vs Function Oriented)

COMPARISON OF SOFTWARE METRICS: SIZE-ORIENTED vs. FUNCTION-ORIENTED

1. SIZE-ORIENTED METRICS – LOC (Lines of Code)



Measure: Count the total lines of code written.





Tech Ex: Project A
10,000 LOC.
Defects found = 50.

Defects/KLOC =
(50 / 10) = **5**
(Calculated in Unit IV!)



5 per KLOC

Biggest Problem: Language dependent.

Python (50 lines)

```
def sentcnr(self):
    try {
        printf("sate logic function",
        print(""You cnant dne dos " + ."))
    }
    return 1
codebaset)
```

Single logic function

Assembler (500 lines)

```
mawtlow xssembler
code:
    cat reqt_cat:crn:$10,10], tsf_entract100
    mov code: 471, n1, sox
    let low_codr cod{$t11 60, assexbler sats100
    mov codo: t61, r33, sox
    let lev_sode cen[8135 5, assembler:scas333d
    cait vaos, weoc:tk, poqic.ledc
```

‘10% LOC but inconsistent

LOC inflates or **deflates** artificially, misrepresenting actual progress.

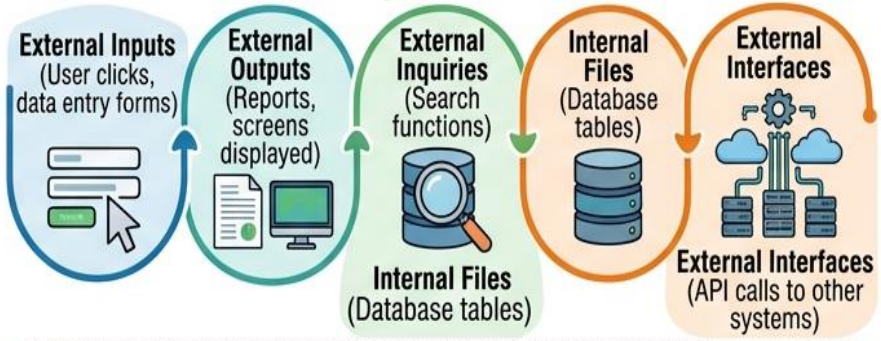


2. FUNCTION-ORIENTED METRICS – FP (Function Points)



Measure: Count the functionality provided to the user, regardless of language.

5 Components of FP



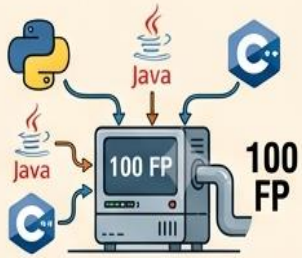
Tech Ex: A ‘Login’ feature



External Input
(User credentials entered, login button pressed)

External Output
(Access granted, redirect to homepage)

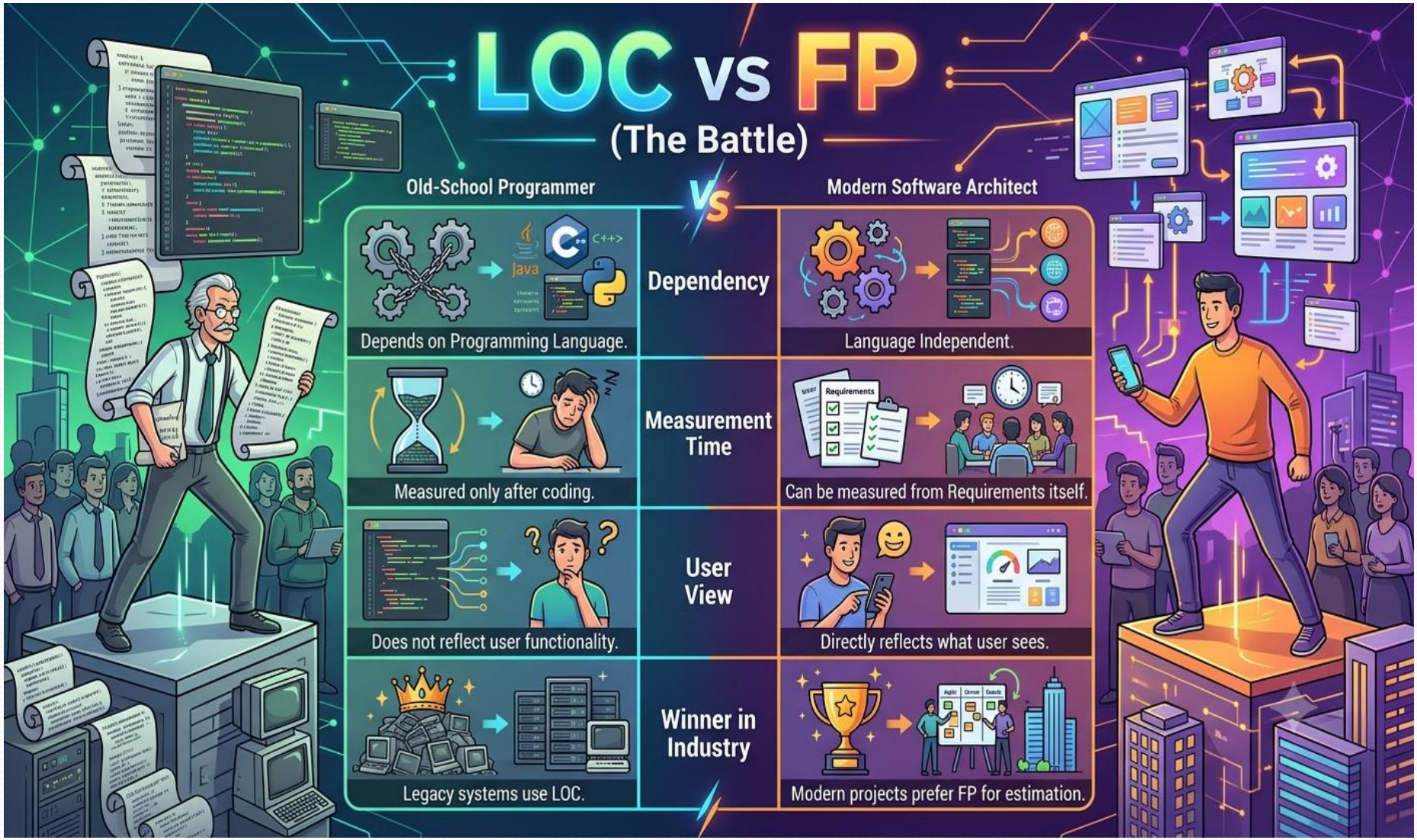
Calculated FP = 100.



100 FP

Even if written in Python or Java, FP remains 100.

FP v/s LOC Battle :



Project Estimation (The Money & Time Math) :

PROJECT ESTIMATION (THE MONEY & TIME MATH)

COCOMO I (CONSTRUCTIVE COST MODEL I) – 1981 CLASSIC

Effort = a * (KLOC)^b (Person-Months) → Effort (Person-Months)

- **KLOC** (Kilolines of Code)
- constants **a, b**

3 Modes

Organic: Small teams, simple requirements (e.g., Attendance system)



Constants: a=2.4, b=1.05

Semi-detached: Medium complexity (e.g., Library Management)



Constants: a=3.0, b=1.12

Embedded: High complexity, strict hardware constraints (e.g., Missile guidance system)



Constants: a=3.6, b=1.20

TECH EXAMPLE

Project: 10 KLOC, Organic → Effort = $2.4 * (10^{1.05})$ → ≈ 27 Person-Months

Project Estimation (The Money & Time Math) :

PROJECT ESTIMATION (THE MONEY & TIME MATH): COCOMO II – MODERN VERSION (2000)

Why new? Because we have **Agile**, **Reuse**, and **Rapid Development** now.

Agile | Reuse | Rapid Development | NOW

3 Sub-models

1 Application Composition



- Uses **Object Points** (screens, reports)



OBJECT POINTS COUNT
(e.g., 50 screens, 15 reports)

↓

Adjustment based on complexity

2 Early Design



- Used when requirements are vague.
- Uses **Function Points** to guess size.



REQUIREMENTS (Vague) → **Function Point Counting** (guesstimation) → **Size Estimate**

3 Post-Architecture



- Used once the system architecture (backend/ frontend) is finalized.
- Uses **KLOC** and **17 Effort**
  Uses **KLOC** and **17 Effort Multipliers** (like Reliability, Platform Difficulty, Team Capability).



ESTIMATED EFFORT = Base KLOC Size x (Multipliers_1 * Multipliers_2 * ...)

Tech Ex:
with real examples

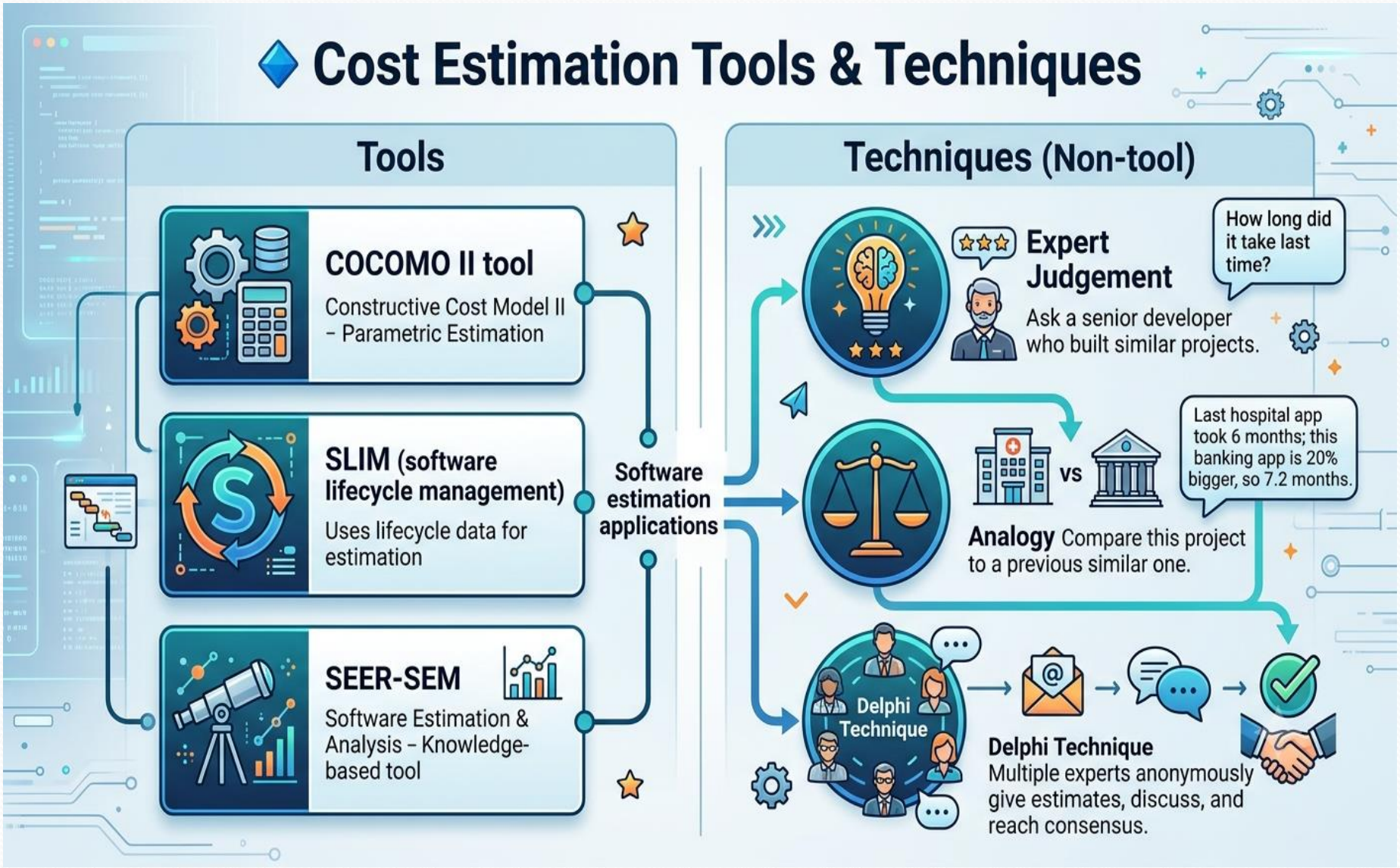
If team capability is "**High**" (multiplier 0.85), effort **reduces**.



If platform is "**Difficult**" (multiplier 1.5), effort **increases**.



Cost Estimation Tools & Techniques



Typical Problems with IT Cost Estimates (Why we always go over budget)

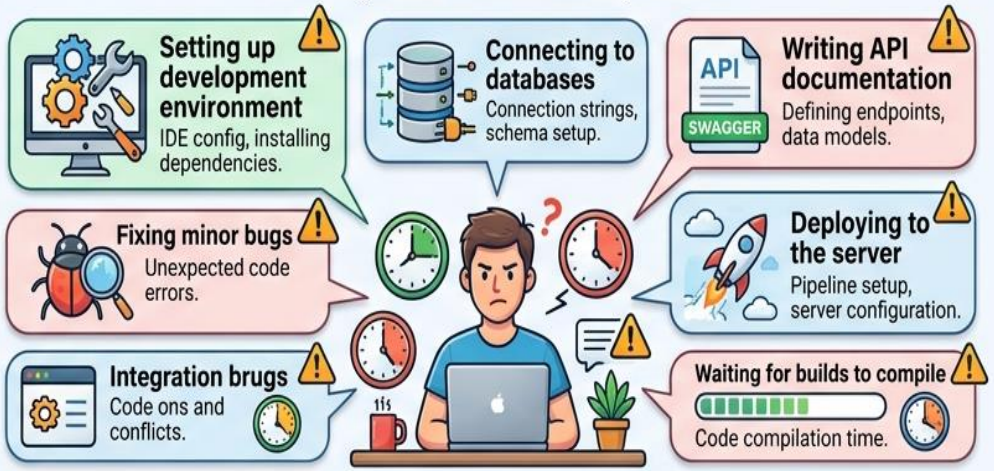
Problem 1: The "Everything Will Go Perfectly" Trap (Over-Optimism)

What Happens: Developer's Optimistic Plan

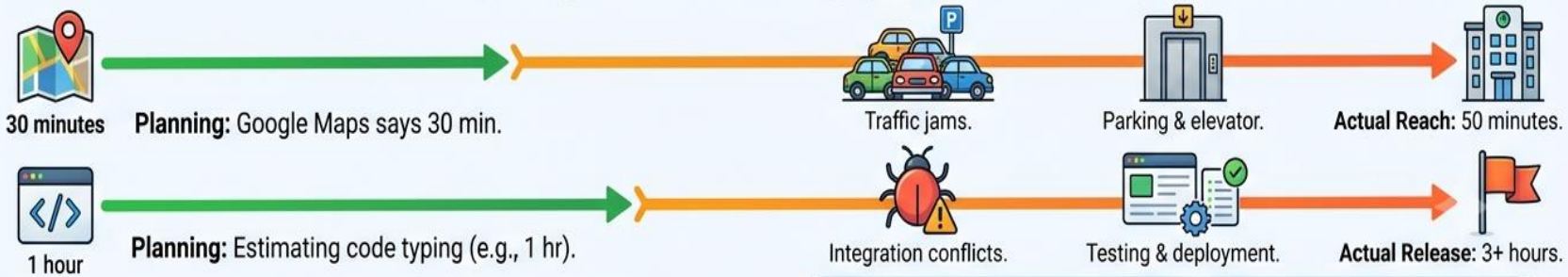
Developers only count the time to **type** the code.



The Forgotten Stuff: Reality Check

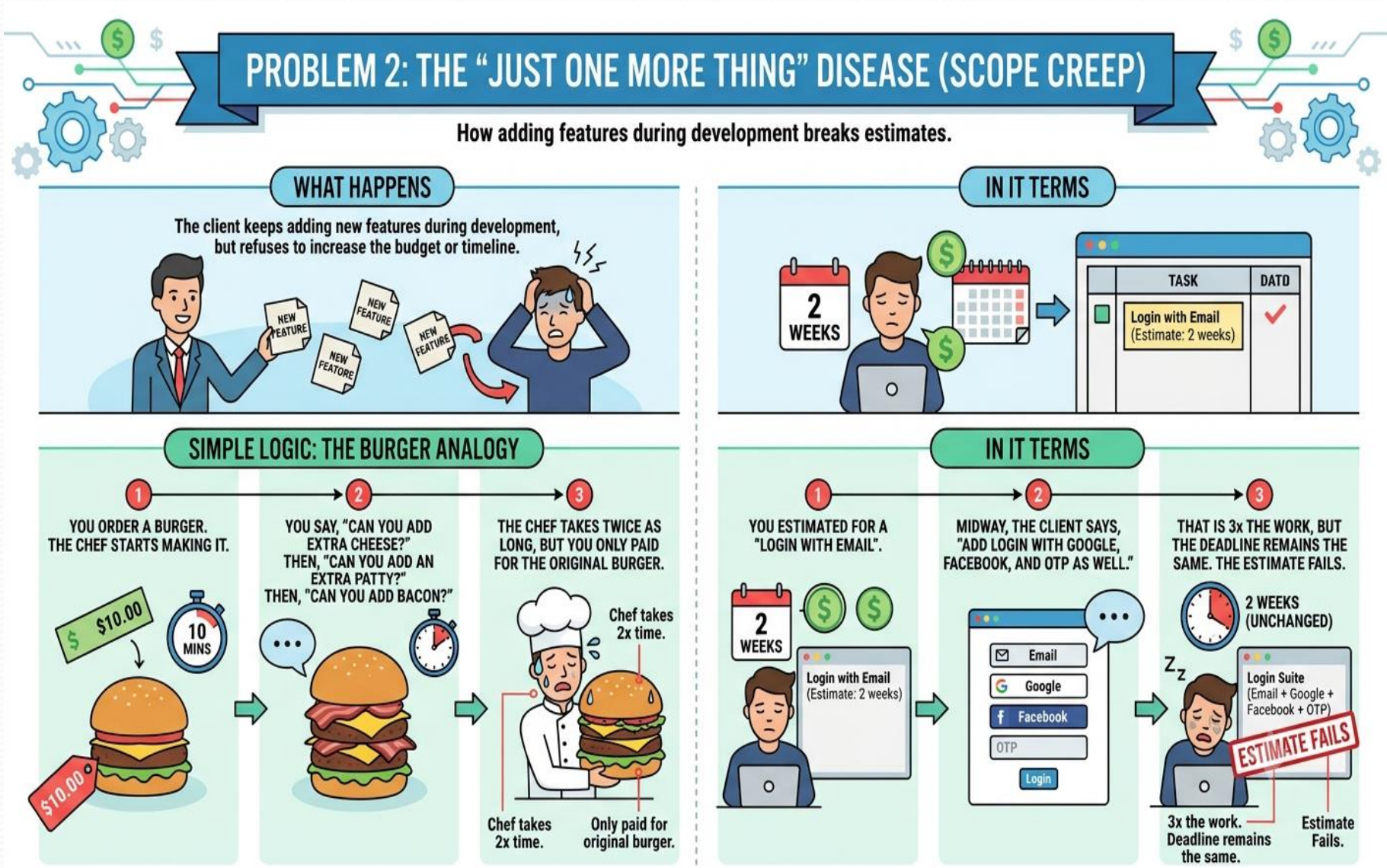


Simple Logic: The Traffic Analogy (Applied to Estimation)

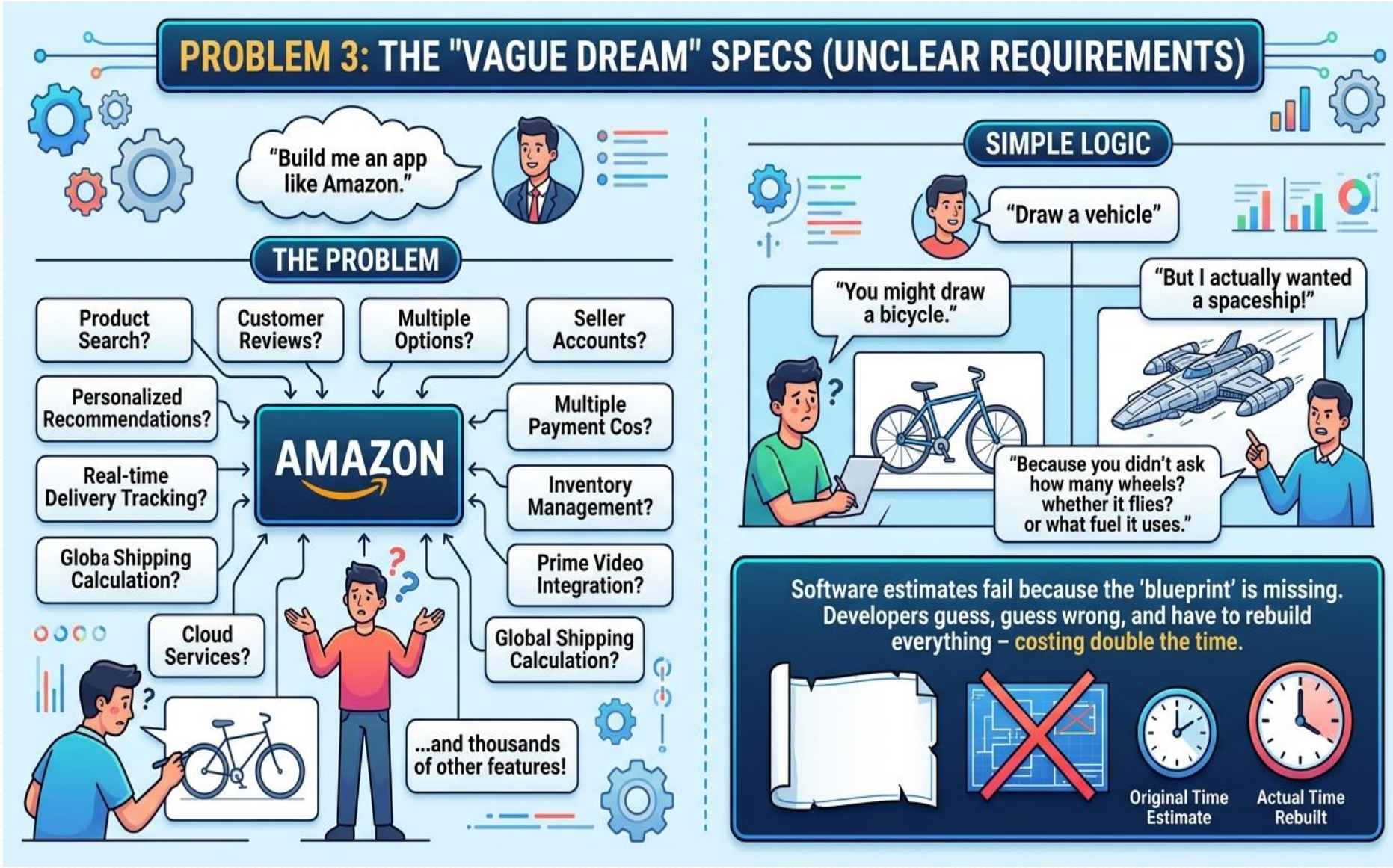


Estimation forgets the "traffic" (**integration, testing, deployment**). **Software** estimation does the same – it forgets the "traffic".

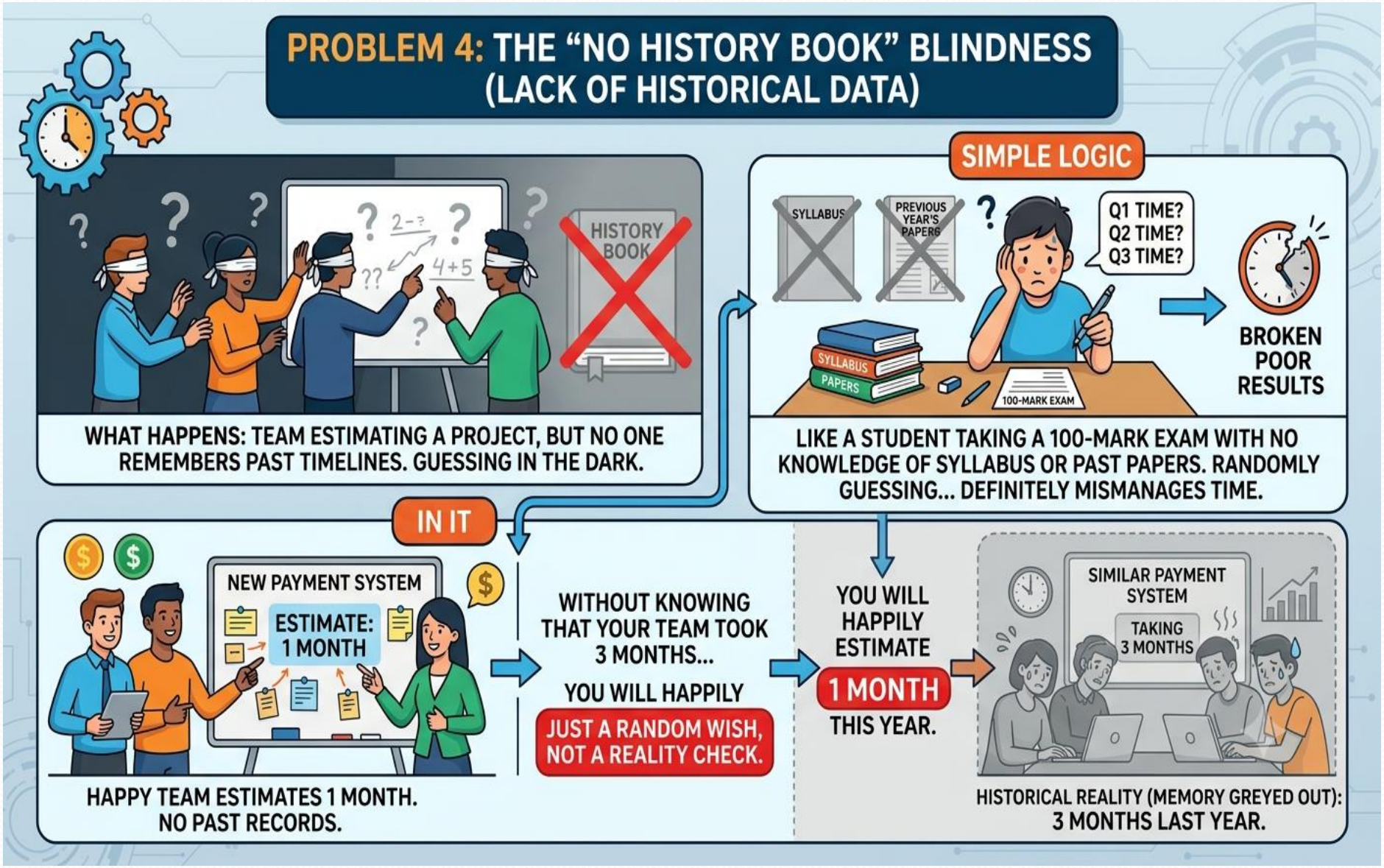
Typical Problems with IT Cost Estimates (Why we always go over budget)



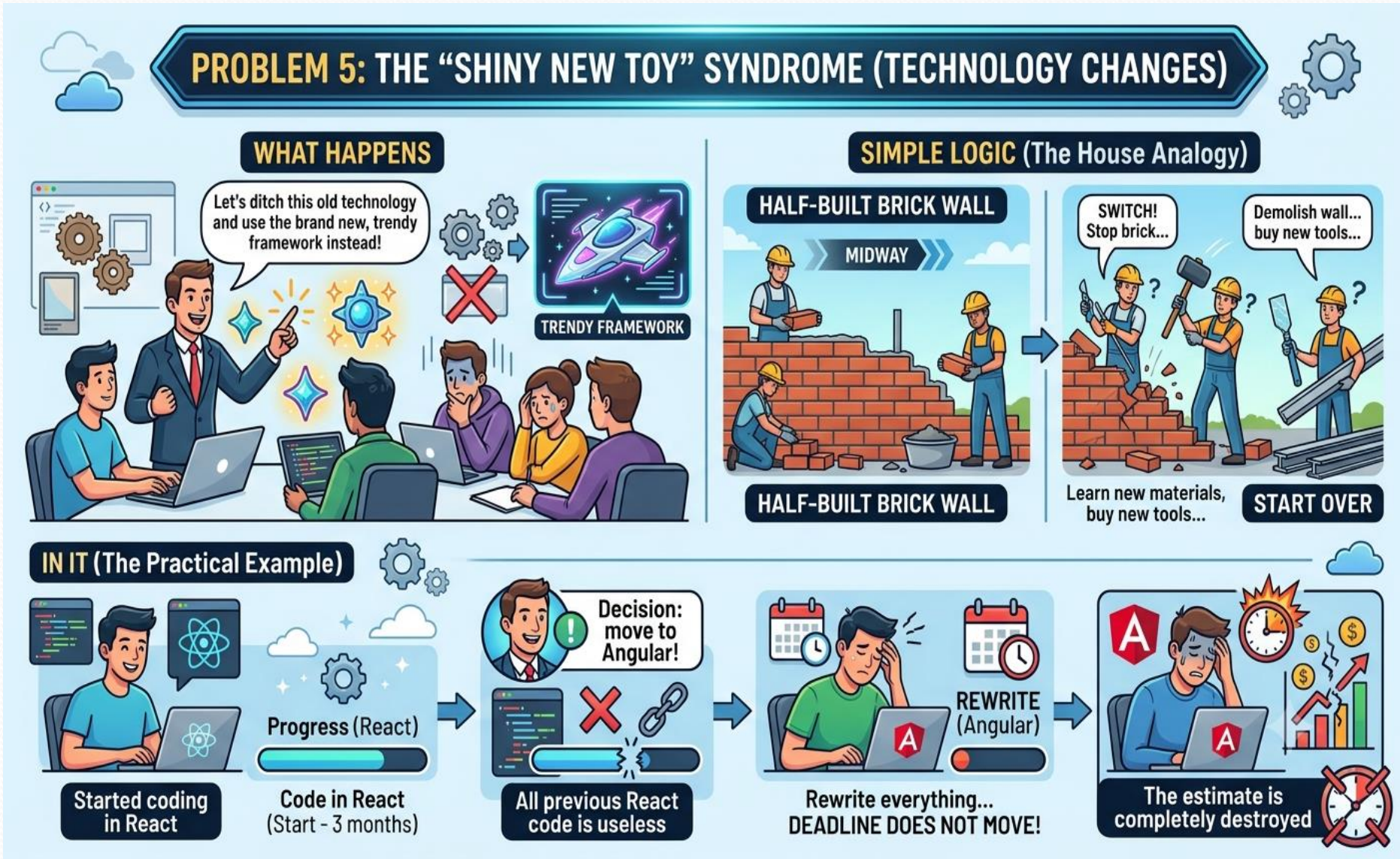
Typical Problems with IT Cost Estimates (Why we always go over budget)



Typical Problems with IT Cost Estimates (Why we always go over budget)



Typical Problems with IT Cost Estimates (Why we always go over budget)



Typical Problems with IT Cost Estimates (Why we always go over budget)

COMMON PROJECT PROBLEMS & THEIR SIMPLE EXPLANATIONS

OVER-OPTIMISM



IN ONE SIMPLE SENTENCE:
We only counted the fun coding part, not the boring setup and fixing part.

SCOPE CREEP



IN ONE SIMPLE SENTENCE:
The client kept adding extra toppings to the pizza without paying extra.

UNCLEAR REQUIREMENTS



IN ONE SIMPLE SENTENCE:
Nobody wrote a clear blueprint; everyone just guessed.

NO HISTORICAL DATA



IN ONE SIMPLE SENTENCE:
We estimated blindfolded because we forgot what happened last time.

TECH CHANGES



IN ONE SIMPLE SENTENCE:
We changed our tools halfway through the race and fell behind.